

Portfolio



DANIEL WLIU

Made in Blender. No AI Generated Graphics.

STUDYING
CS AND FINANCE, WATERLOO

SITE
[DANIELWLIU.COM](https://danielwliu.com)

GITHUB
[DANIELWLIU07](https://github.com/danielwliu07)

LINKEDIN
[IN/DANIELWLIU](https://in/danielwliu)

LINES OF CODE 95,500

BUILD TIME 36 HRS

TEAM 4 PEOPLE

An undo button for your room.

It keeps your room in git, the tool programmers use to save versions of their code. It notices when things move, plans how to put them back, and when something goes wrong it **files its own bug report**. Built in 36 hours because the idea would not leave me alone.

MADE AT
Hack the North 2026

TEAM
Four

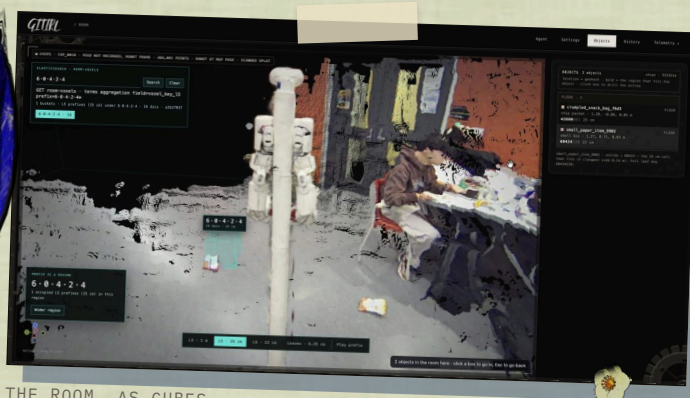
MY PART
Robot vision, search, the website



THE ROBOT, CHECKING THE ROOM



WHAT THE ROBOT SEES: TWO CAMERAS, LIKE TWO EYES, FOR DEPTH



THE ROOM, AS CUBES

HOW IT WORKS

SEEING

A map made of 3 cm cubes

The robot builds its own 3D map from two cameras. Anything rising 9 cm above the floor becomes an object, and the camera names only what it recognises; the rest stays "unknown" rather than guessed.

NOTICING

Seen twice before it counts

Scanning works like checking for unsaved changes. A move only counts once two separate passes agree, so a hand passing through is not a mess.

UNDOING

A plan that never collides

Undo becomes an ordered tidy-up plan that never sets something down on an occupied spot; swaps set one object aside first. The real robot drove and pointed; picking up ran in simulation.

Every save is a snapshot in git.

The room lives in a real git project, one small text file per object, so "what moved?" is a side-by-side comparison of two saves, and undo is rolling back to an earlier one. **Git** writes the room down; **Elasticsearch**, a search engine, answers questions about it.

HACKATHON PAGE

devpost.com/software/house-bot

GITHUB

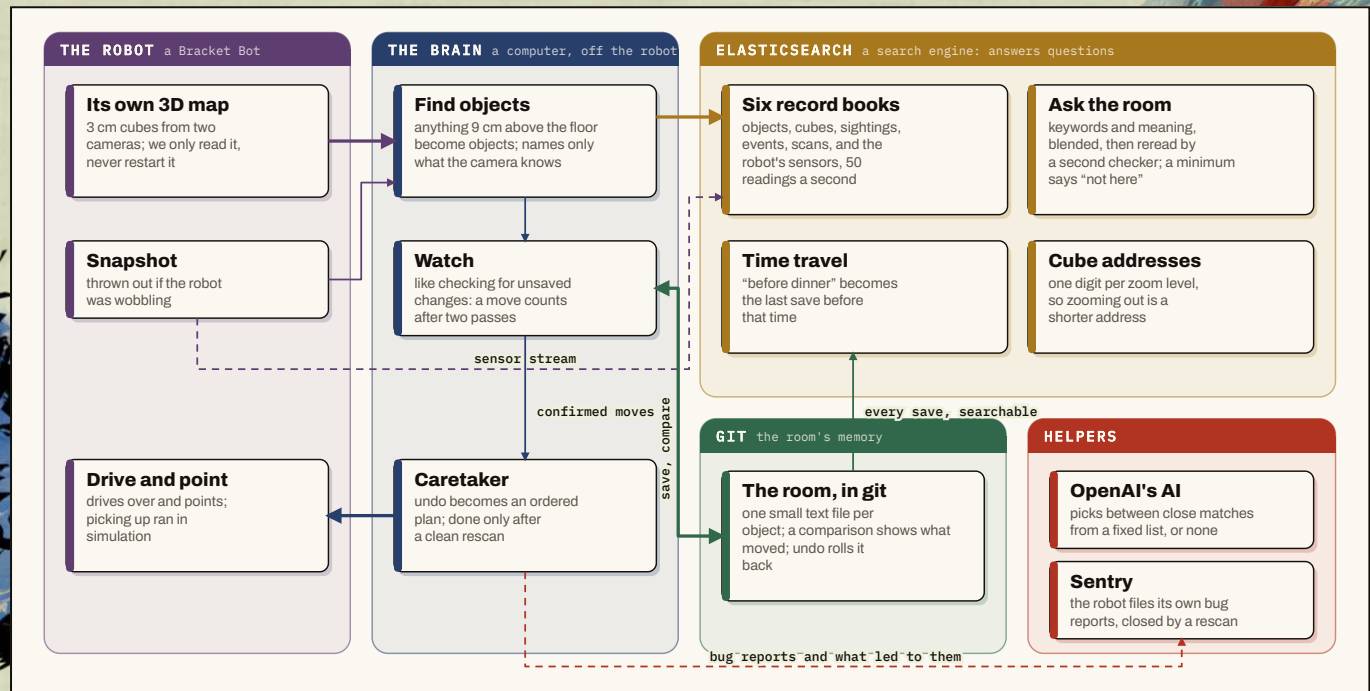
github.com/DanielWLIU07/hack-the-north

LIVE

gitirl.health

DEMO

youtu.be/FEGR3mJmXMg



6.9M

robot sensor readings, summed up in 0.03 seconds

6

searchable record books, each linked to the bug reports

60%

of a camera frame must match the map before it may name anything

2

separate passes before a move counts

SEARCH

Keywords and meaning, blended

Ask for "the thing I cut paper with" and Elasticsearch runs two searches at once: keyword matching, and meaning search, which turns text into numbers so similar ideas sit close. The lists are blended, then a reranker, a second AI, rereads the top 50 against your question.

NOT FOUND

Teaching search to say no

Meaning search always returns its nearest guess, so I measured a minimum score. Below it the answer is "not here"; in the grey zone the robot asks first. "Pick up the trash" is refused instead of grabbing a cup.

FILING

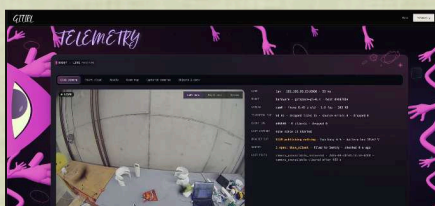
An address that zooms

Each cube's address is a string of digits, one per level of halving the room, so zooming out is reading a shorter address. Elasticsearch works out the bigger boxes itself as each cube arrives.

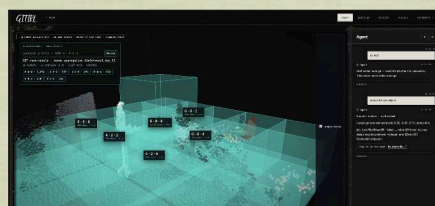
TRACING

From a bug to the moment behind it

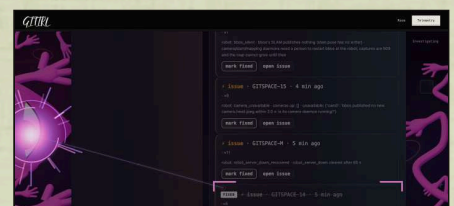
Every record carries the same tag as its report in Sentry, a service that collects bugs, so one jump goes from a failure to the scan behind it and the robot's sensor readings in the seconds before.



LIVE SENSOR READINGS



ASK THE ROOM A QUESTION



BUGS IT FILED ITSELF

LINES OF CODE 38,500

BUILD TIME 36 HRS

TEAM 4 PEOPLE

A fruit-picking robot you drive from a browser.

It finds fruit, decides if it is ripe, picks it and sorts it. Seeing and deciding run on one chip, a small computer; wheels, arm and brakes run on a second, simpler chip that reacts instantly, so the **emergency stop still works** when everything else falls over. At the event, a browser tab was the controller. Try the simulator at pomme.live

AWARD

Qualcomm's Build at the Edge prize

MADE AT

Hack the 6ix 2026

TEAM

Four

MY PART

Software, camera vision, laser mapping, design

POMME, ARM UP, READY TO PICK

UNDER THE HOOD

SEEING

Ripeness is colour

I trained the fruit detector on computer-drawn pictures of our 3D-printed fruit. One pass names the fruit and whether it is ripe, and I switched off colour-shifting in training, because here ripeness literally is colour.

DECIDING

One step at a time, 20 times a second

Roam, find fruit, drive up until it fills a tenth of the view, centre it, pick, sort, drop. Each step is checked 20 times a second and never holds up the next.

MAPPING

A map from a spinning laser

I wrote the mapping, which uses lidar, a laser that spins and measures distance. My first version decided a sensor sitting perfectly still had wandered 300 metres, so the rewrite throws out any match that implies an impossible jump.



THE PRINTED ARM

FIRST DRIVE

Open a tab. Drive a robot.

The website is the cockpit. A live map, the robot's own view, every pick counted, and a PlayStation controller **read straight from the browser**. No app to install. The public site runs a simulated robot, so anyone can try it.

HACKATHON PAGE

devpost.com/software/autoagrinator

GITHUB

github.com/DanielWLi07/pomme

LIVE

pomme.live



9

pages in the dashboard

5

status updates a second from the robot

30

steering updates a second from the controller

3.2 MB

fruit detector running in your browser, shrunk from 11.6 MB

LANDING

A painted orchard, a real robot

The rover rolling in is a Gaussian splat: a 3D capture rebuilt from ordinary phone video of the real robot, placed in a scene filtered to look hand-painted.

IN YOUR BROWSER

The detector, shrunk

Storing the detector's numbers with fewer digits made it a quarter of the size and 2.6 times faster, but broke its sense of how sure it is, which the robot needs. So the small copy runs on the site, labelling every harvest photo.

THE ROOM

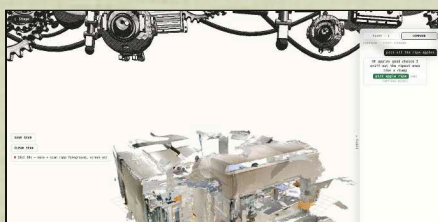
Click a TV, tune in

The dashboard is a dark room of old TVs. Click one, the screen fills with static, the camera dives in, and that page tunes in behind it.

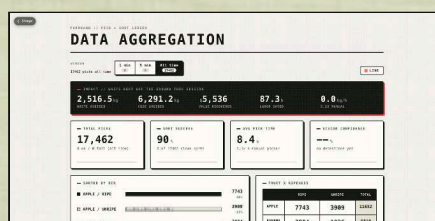
DRIVING

Hold the trigger to move

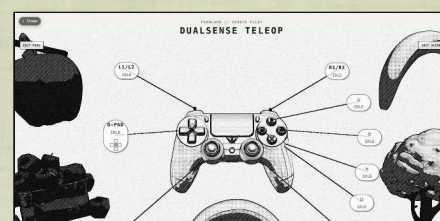
A PlayStation controller works straight from the tab. Steering goes out 30 times a second, the wheels only move while you hold the right trigger, and driving skips the hub, the server that relays everything else.



WHAT THE ROBOT SEES, LIVE



EVERY PICK, COUNTED



DRIVE IT WITH A CONTROLLER

Two brains, one robot.

One board, two chips. The smart one, a small computer, sees the fruit and decides what to do. The simple one moves the motors and owns the brakes, so if the smart half crashes, the robot still stops.

THINKS

20 times a second

ARM

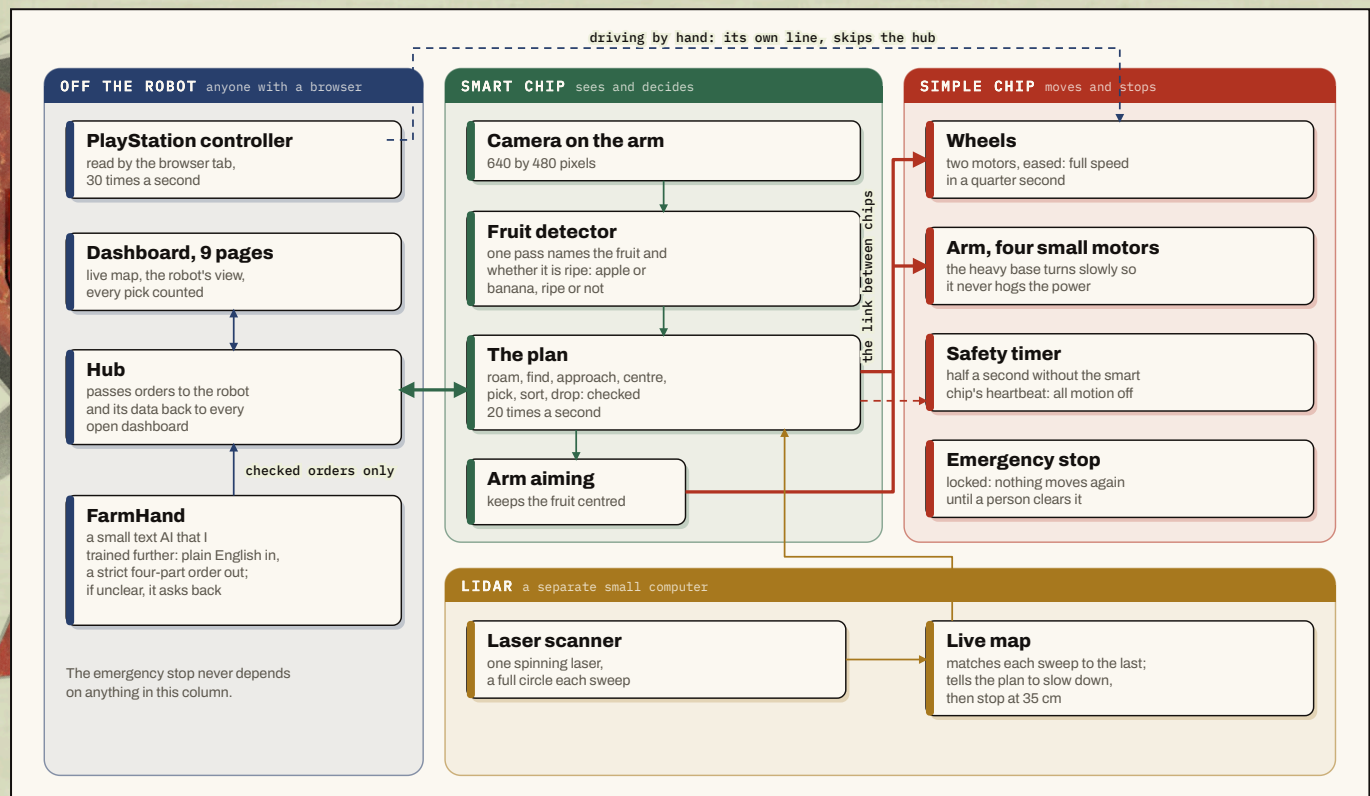
Four small motors

MAP

From a spinning laser

SIGNAL LOST

Motion off in half a second



SAFETY

The brakes answer to no one

If the smart chip stops saying "still here" for half a second, the simple chip cuts the wheels by itself. A stop stays locked on until a person clears it.

LANGUAGE

FarmHand, my small text AI

I fine-tuned a small text AI, giving it extra training on 2,349 made-up commands. On 58 messy ones full of typos and slang it got 81% right; hand-written rules got 47%.

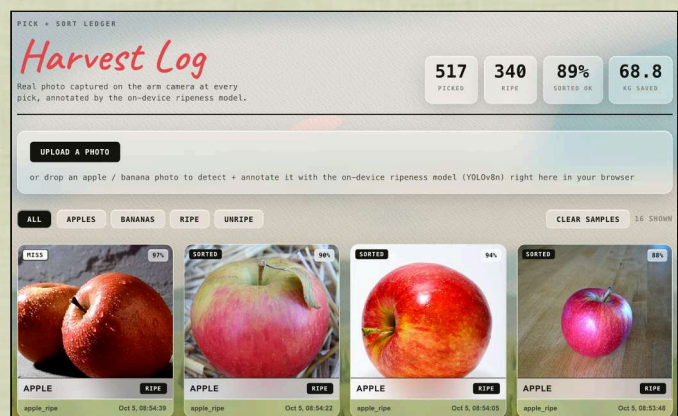
GUARDRAIL

Only strict orders get through

Anything FarmHand says that does not fit a strict four-part order is thrown away before it reaches the robot. If a request is unclear, it asks back instead of guessing.



THE TV ROOM: PICK A SCREEN



THE HARVEST LOG: EVERY PHOTO LABELLED RIPE OR NOT BY THE DETECTOR, IN YOUR BROWSER

LINES OF CODE 51,300

BUILD TIME 10 MONTHS

TEAM SOLO

My personal website, one look per page.

danielwliu.com is where I keep my work. Instead of a template, every page is its own little world, made in Blender, free 3D modelling software, and brought into the browser, and the tour ends at my resume, **dealt to you across a poker table**.

BUILT
SoloLIVE
danielwliu.comCODE
[GitHub](#)

1 LANDING

2 ABOUT

3 EXPERIENCE

4 PROJECTS

5 RESUME

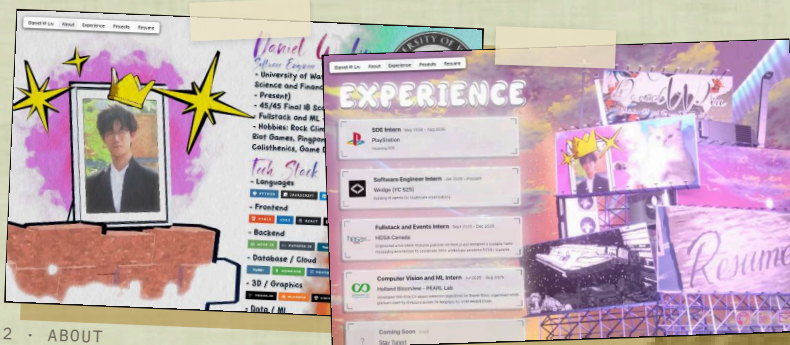


1 • THE LANDING, IN THE SAME INK AS THIS BOOK

2, 3, 4 • EVERY PAGE ITS OWN SET

About fills in like wet watercolour. Experience puts each job on a billboard in 3D. Projects opens like a manga, then turns into a carousel of cards that play video.

Then the finale: the resume is a casino you can actually play. Turn the page for how it gets to your screen.

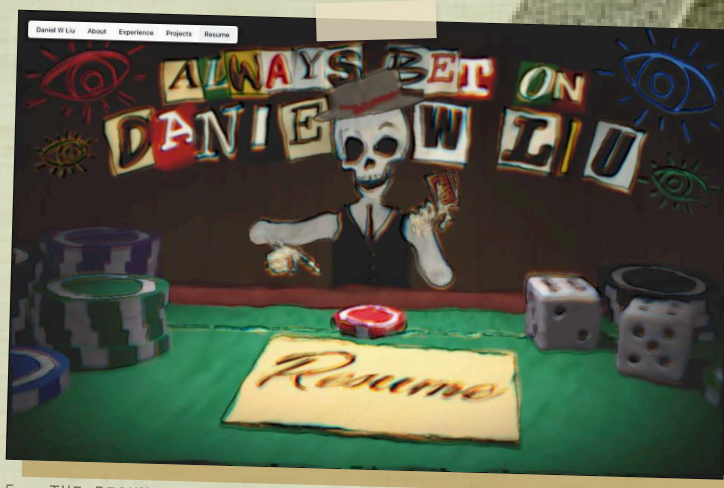


2 • ABOUT

3 • EXPERIENCE



4 • PROJECTS, A MANGA CAROUSEL



5 • THE RESUME: A SKELETON DEALER DEALS IT TO YOU AS A CARD

How the casino gets to you.

The resume is a 3D casino, and not every laptop can draw it. So I built a way to start by watching it as live video from a graphics server, the way cloud gaming works, while your own computer, if it is strong enough, gets ready behind it and takes over before the show starts. It is a working prototype that one setting switches on. If anything about streaming fails, you get the normal page: **it can only make things faster, never broken.**

GITHUB

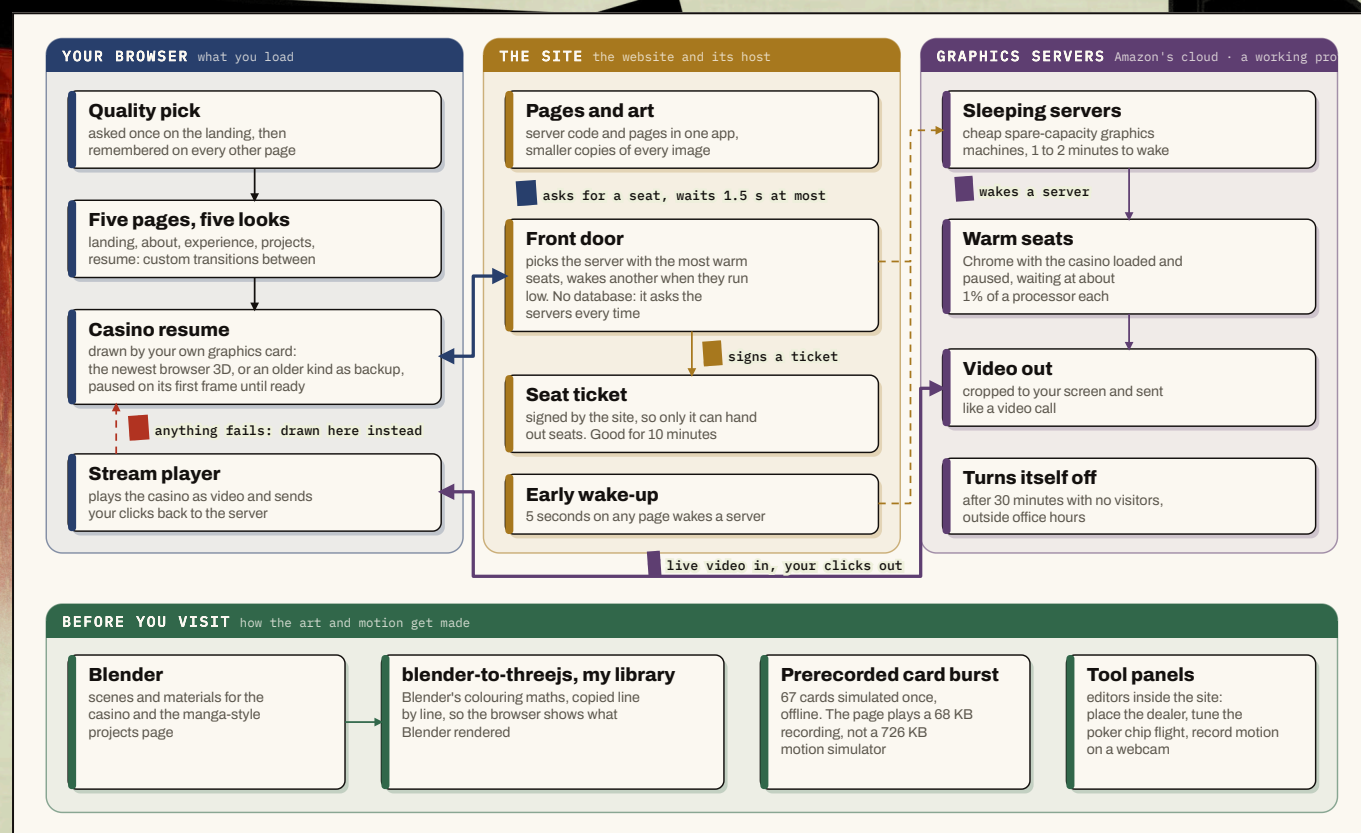
github.com/DanielWLIU07/daniel-w-liu-portfo
[lio-website](https://github.com/DanielWLIU07/daniel-w-liu-portfo)

MY BLENDER-TO-BROWSER LIBRARY

github.com/DanielWLIU07/blender-to-threejs

BUILT

Solo



0.3 s

to the first streamed picture, measured on my laptop

0.94 → 0.18 s

the longest freeze while loading, before and after my fixes

0

databases needed: the site just asks the servers who is free

30 min

with no visitors and a server turns itself off

SHADERS

Blender, matched exactly

Browsers and Blender do lighting maths slightly differently, so a scene can load fine and still look wrong. My library, blender-to-threejs, copies Blender's own shader code, the maths that colours each pixel, to the browser so the two match.

STREAMING

A seat that is already warm

Opening any page quietly wakes a graphics server in the cloud. By the time you reach the resume, a seat is waiting. Your ticket to it is signed, so it cannot be forged, and expires in 10 minutes.

FALLBACKS

It can only make things faster

No free seat, a slow answer or a dropped video all end the same way: the casino is drawn by your own computer's graphics chip instead.

BUILT WITH

Next.js: the website

Three.js: 3D in the browser

WebRTC: live video

Amazon cloud servers

MediaPipe: webcam tracking

Blender

LINES OF CODE 28,900

BUILD TIME 4.5 MONTHS

TEAM SOLO

A world made of cubes, in four dimensions.

The world has four directions, not three. What you walk around in is a 3D slice of it, and scrolling turns the slice. The camera never moves, yet the landscape rebuilds itself, because **you are looking at a different cut of the same world.**

BUILT

Solo, from scratch

STARTED

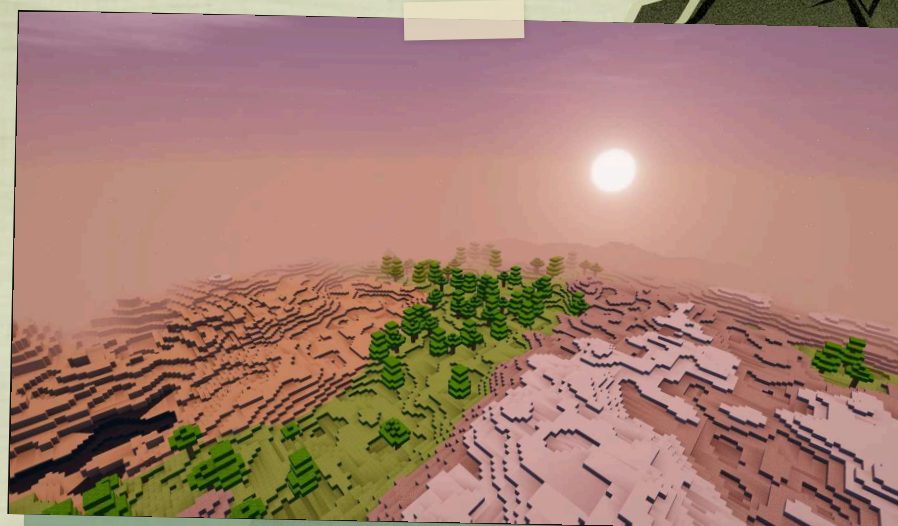
May 2026

WRITTEN IN

C++, with OpenGL for graphics

EVERY PIECE

A cube, or a slice of one



ONE SLICE OF A FOUR-DIMENSIONAL WORLD

TURN THE CUT. SAME WORLD, SAME CAMERA.

TURNING

A fifth of a degree a notch

Each scroll turns the cut about a fifth of a degree into the fourth direction. The cut turns around you, so the ground under your feet stays put while a lake with a treeline becomes a snowfield.

TERRAIN

Hand-written 4D patterns

The pattern code for shaping hills that I started with stops at three dimensions, so I wrote the 4D version: each sample blends the 16 corners of a tesseract, a 4D cube. Near the surface, a second 4D pattern carves overhangs.

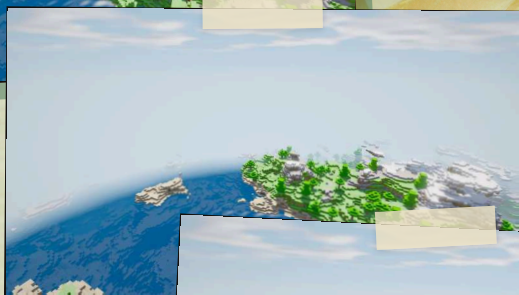
SKY

Nobody places the rainbow

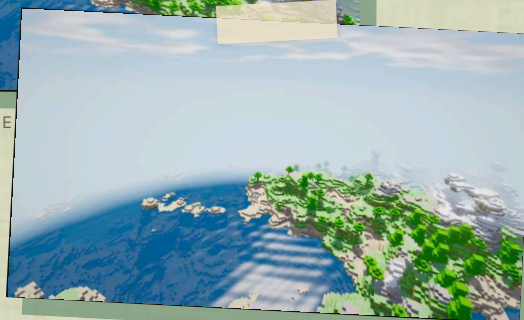
The sky's drawing code places it 42 degrees from the point opposite the sun, so it rises by itself at sunset. 9,000 fireflies each work out their own flight path on the graphics chip.



FLAT



TURNED A LITTLE



TURNED TWICE AS FAR



RAINBOW



AURORA



FIREFLIES



UNDERWATER

How a 4D world gets drawn.

Nothing four-dimensional is stored. The world is a formula: every patch of ground is worked out fresh for the angle you look through. Background helpers build it while one main helper draws, and flat faces get merged so there is far less to draw, **even when the slice is tilted**, which the engine's own design notes once said was impossible.

GITHUB

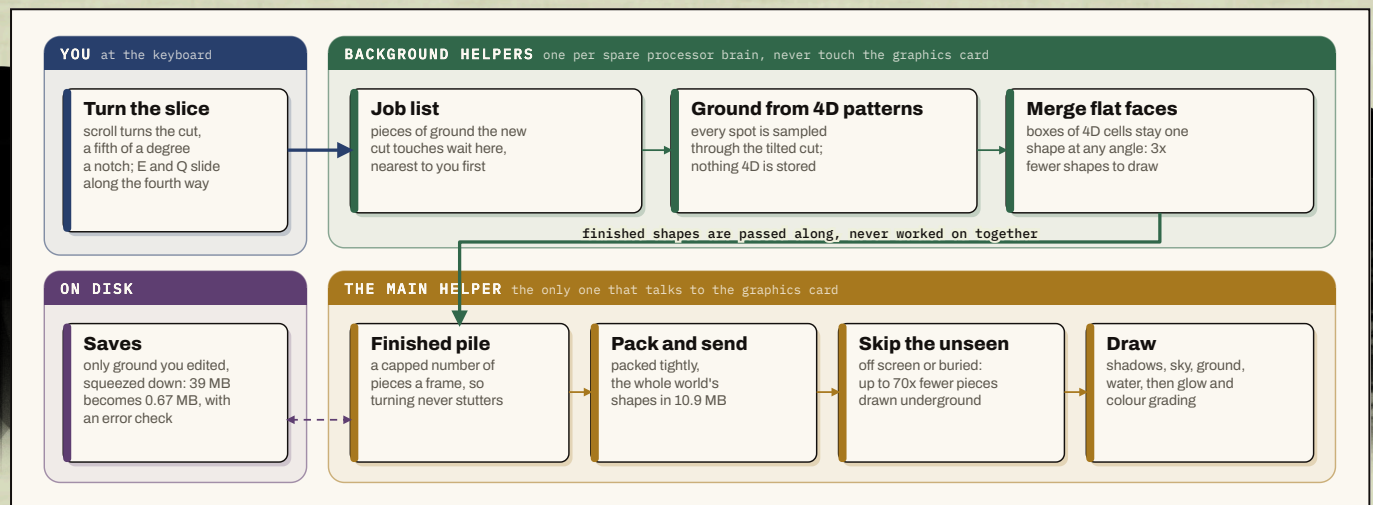
github.com/DanielWLi07/voxel-engine

RUNS ON

Mac, Linux, Windows

BUILT

Solo



0.008 s

to draw each picture of a 277-million-block world, in its ordinary 3D mode

11.6x

less memory for the world's shapes, by merging and packing them tightly

3x

fewer shapes for the graphics card to draw

58x

smaller saves, 39 MB to 0.67 MB

SHAPES

A block is a slice, not a cube

Turn the cut one way and every block stays four-sided. Turn it two ways at once and triangles, pentagons and hexagons appear. The walls of a 4D cell can carve at most six sides, so a block is never more than a hexagonal pillar.

MERGING

Big flat faces, even on a tilt

Neighbouring block faces get merged into big rectangles, so the graphics card has far fewer shapes to draw. On a tilted cut it only works if I merge in the 4D world, not the flat picture.

TEAMWORK

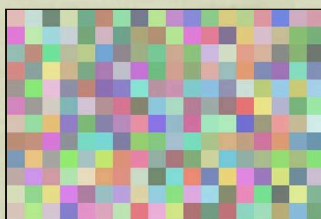
Many builders, one painter

A computer's processor has several cores, small brains that work at once. All but one build ground in the background, each with its own copy of what it needs, then hand it over. Only one touches the graphics card, so nothing fights over it.

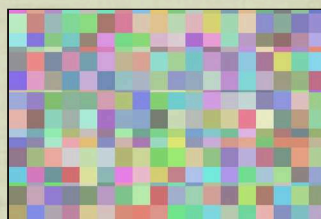
TESTS AND SAVES

Turn it, turn it back, check every block

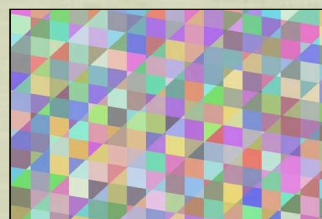
Turn the cut and back, and a fingerprint of every block must match exactly; an automatic check runs every time I change the code. Saves store "stone, forty in a row" instead of forty stones, and a built-in error check rejects corrupted files.



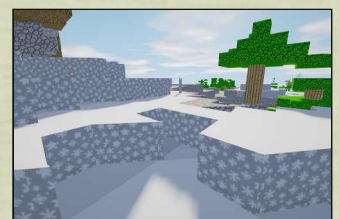
CELLS, FLAT CUT



ONE PLANE TURNED



BOTH PLANES: HEXAGONS



A BLOCK IS A SLICE, NOT A CUBE

LINES OF CODE 12,500

BUILD TIME WEEKEND + 1 MONTH

TEAM 4 PEOPLE

It guesses where you are from a photo.



THE LANDING PAGE

Drop in a picture of anywhere and it works out where on earth it was taken, the way a geography nerd would: road lines, scripts on signs, licence plates, plants, the shape of the hills. Then it **shows its working**, so you learn the clues too.

AWARD

Best UI Hack

OUT OF

124 projects

TEAM

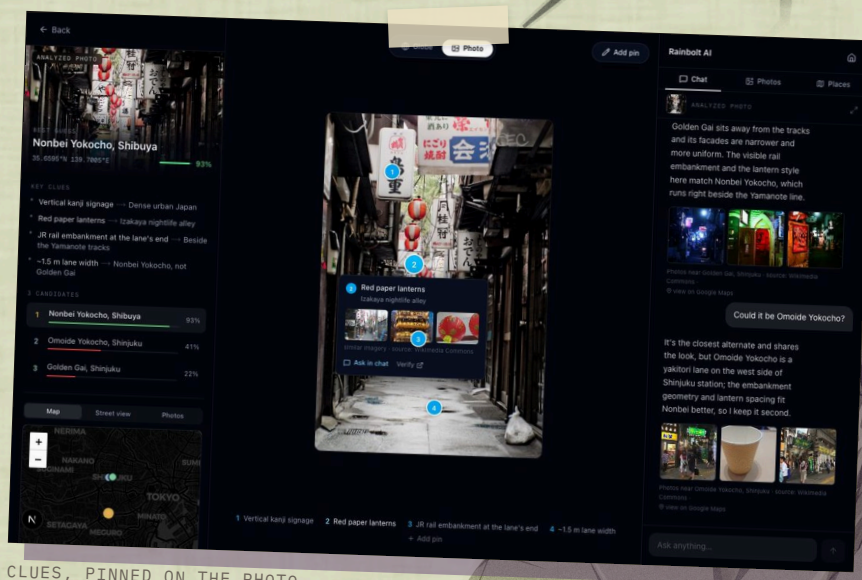
Four

MY PART

The globe, clue pins, the rebuild

WHAT I BUILT

- The 3D globe.** About 26,000 glowing dots drawn by my own graphics code, with a denser layer that fades in as you zoom. Dots now double in size up close, so coastlines read as solid land.
- Clue pins.** A second AI pass places 3 to 6 numbered pins on the exact spots in your photo, each with reference pictures pulled from Wikimedia.
- A chat that does not lose its place.** If the AI stalls mid-answer, it keeps what it has said and hands off to a lighter backup AI.
- The rebuild.** When the photo library was deleted, I rebuilt it piece by piece up to the free plan's monthly limit, and rewrote the 242 clues. Sign-in moved from two services to one, plus guest mode and a tutorial.



CLUES, PINNED ON THE PHOTO

WHY IT IS A GAME

Guessing places from a photo is already a game people play for fun. Rainbolt plays it with you and explains every move, so **each clue it spots is one you learn to spot yourself**. The next photo, you get there first.

One photo in, one pin out.

The photo becomes a fingerprint of 512 numbers, matched against location-tagged photos and against clues written in plain words. An AI reasons through the evidence out loud, places its guesses on the globe, and **street photos let you check its pick yourself**.

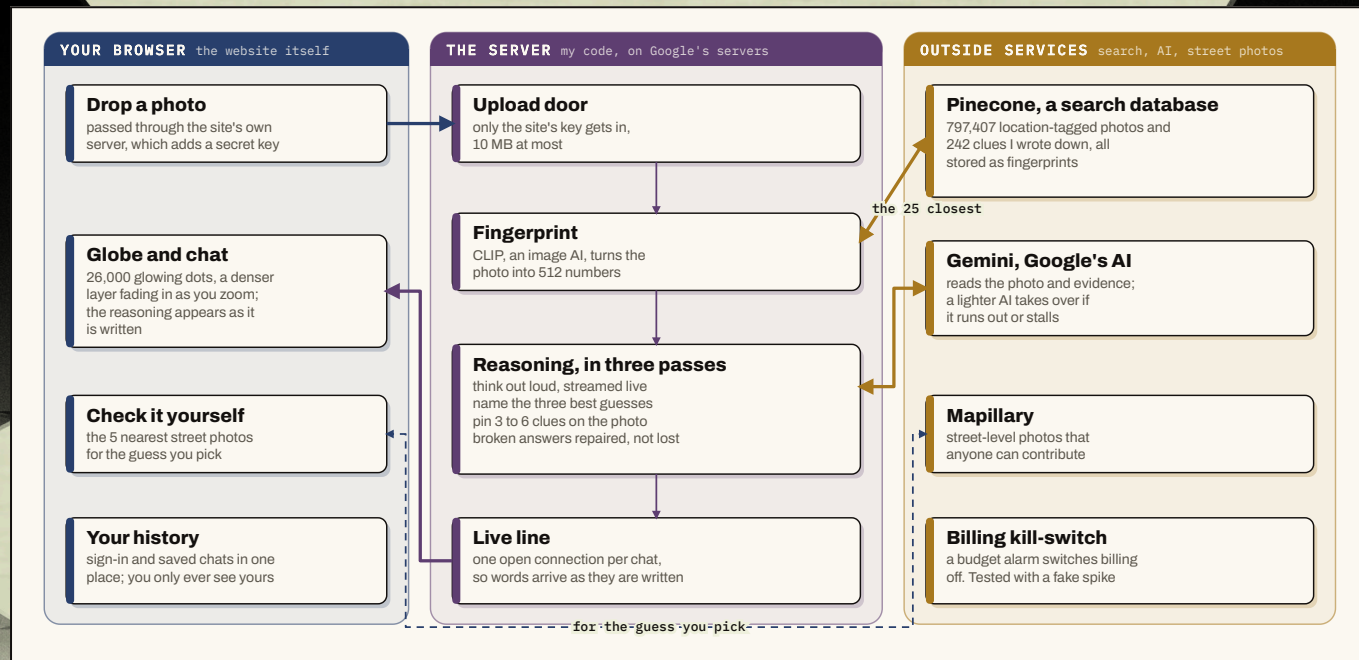
HACKATHON PAGE

devpost.com/software/rainbolt-ai

GITHUB

github.com/dtpu/rainbolt.ai

LIVE

rainboltai.vercel.app**797,407**

location-tagged photos in the library I rebuilt

242

location clues, from plates to bollards

1st

Best UI (user interface), out of 124 projects

3

AI passes per photo: think out loud, name three guesses, pin the clues

MATCHING

Close fingerprints, close places

CLIP, a free image AI, runs on the server and turns your photo into 512 numbers. Pinecone, a search database, returns the 25 closest photos, keeping only strong matches. I do not average the matches: the AI weighs the raw evidence itself.

REASONING

Answers that survive bad replies

The AI sometimes answers in a garbled format, so every reply goes through three readers in turn: strict, forgiving, then auto-repair. A stray apostrophe no longer loses a guess.

CLUES

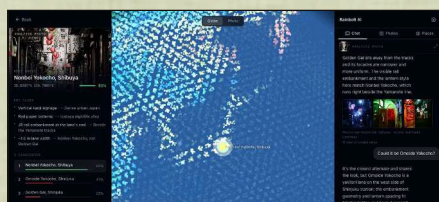
Pictures compared with words

Each clue is a sentence, like the shape of a country's bollards, the short posts beside the road. On a 0 to 1 scale, photo-to-photo matches score around 0.7, photo-to-words only about 0.2, so I gave the clue search its own, much lower bar.

SAFETY

A bill that cannot run away

Uploads need a key only the site's server holds. If spending spikes, a budget alarm triggers a small cloud program that switches billing off. I tested it by faking an overspend.



REASONING, WITH SOURCES



GUESSES ON THE GLOBE



EVERY PHOTO YOU HAVE TRIED

Things I made for no reason.

No brief, no client, no deadline. I wanted to see if I could, and then I wanted to see if I could **do it better**.

BRIEF
None

MADE IN
Blender and Unity

IN BLENDER scenes, a walking goose and two looks



The ink landscape

The scene behind this book's cover: mountains, plum blossom and a red moon, rendered to look brushed in ink. Here it is open in Blender, with the colour recipe, a web of nodes, that makes 3D look painted.



A goose that walks

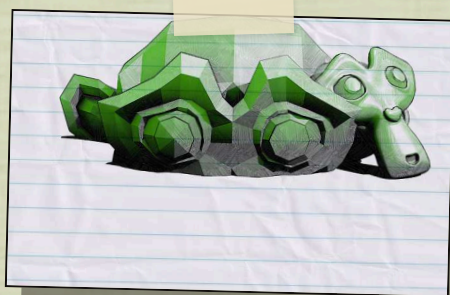
Modelled in Blender and given a skeleton so it can move, then brought into my website, where it walks around a village green with its feet planted on the ground. Its neck follows your pointer, and a honk draws comic squiggles beside its bill.



A website you walk into

The shared student website, a webbing, for Waterloo's Computing and Financial Management program. I made its scenes in Blender as the backgrounds and opening film.

uwaterloocfm.com



Pencil on notebook paper

A second look, nothing like the ink: any model drawn as pencil shading on creased, lined paper, tried on Blender's test monkey.

IN UNITY two games, in a free game engine



Knockout Golf

A same-room party game where you are the golf ball: drag back, aim, launch. No waiting between shots, so everyone is always crashing into each other. Knock them off the stage or race them to the hole. Models from Blender, swaying grass made in Unity's effects editor.

[GitHub](#)



Labyrinth of Doom

Online multiplayer in the spirit of Among Us. Everyone gets a torch in a pitch-dark maze, except the Minotaur, who sees in the dark and hears you walk. Light six statues to put it to sleep, but each one gives you away. A two-week game jam, a build-it-fast sprint, for school; Stack Overflow and YouTube carried.

itch.io

I build things nobody asked for.

More at danielwliu.com.

NAME	DANIEL W LIU
EMAIL	D347LIU@UWATERLOO.CA
SITE	DANIELWLIU.COM
CODE	GITHUB.COM/DANIELWLIU07
LINKEDIN	LINKEDIN.COM/IN/DANIELWLIU

